

iOS面试题合集（内存管理篇）

iOSer iOSer 5月19日

收录于话题

#BAT大厂面试题合集

9个

欢迎点击上方蓝字“**iOSer**”关注我们
再点击右上角“...”菜单，选择“**设为星标**”
我们一起精进、成长！

本栏目将持续更新--请iOS的小伙伴关注我们!

做这个的初心是希望能巩固自己的基础知识，

当然也希望能帮助更多的开发者，

可以加入我们的技术交流群：**834688868**，群里每天都会有干货、技术动向、职业生涯、行业热点、职场趣事等一切有关于程序员的内容分享。

不管你是大牛还是小白都欢迎入驻，分享BAT，阿里面试题、面试经验，讨论技术，大家一起交流学习成长！

目录：

- 1.什么情况使用weak关键字，相比assign有什么不同？
- 2.如何让自己的类用copy修饰符？如何重写带copy关键字的setter？
- 3.深拷贝与浅拷贝分别是什么？
- 4.@property的本质是什么？ivar、getter、setter是如何生成并添加到这个类中的？
- 5.@protocol和category中如何使用@property
- 6.使用CADisplayLink、NSTimer有什么注意点？BAD_ACCESS在什么情况下出现？
7. iOS内存分区情况
- 8.iOS内存管理方式
- 9.循环引用
- 10.ARC 的 retainCount 怎么存储的？

11. `ARC` 在编译时做了哪些工作？

1. 什么情况使用weak关键字，相比assign有什么不同？

什么情况使用 **weak** 关键字？

在 ARC 中,在有可能出现循环引用的时候,往往要通过让其中一端使用 weak 来解决,比如: delegate 代理属性

自身已经对它进行一次强引用,没有必要再强引用一次,此时也会使用 weak,自定义 IBOutlet 控件属性一般也使用 weak;

当然,也可以使用strong。在下文也有论述:《IBOutlet连出来的视图属性为什么可以被设置成weak?》

不同点:

weak 此特质表明该属性定义了一种“非拥有关系” (**nonowning relationship**)。为这种属性设置新值时, 设置方法既不保留新值, 也不释放旧值。此特质同assign类似, 然而在属性所指的对象遭到摧毁时, 属性值也会清空(nil out)。

而 assign 的“设置方法”只会执行针对“纯量类型” (scalar type, 例如 CGFloat 或 NSInteger 等)的简单赋值操作。

assign 可以用非 OC 对象,而 weak 必须用于 OC 对象

2. 如何让自己的类用copy修饰符？如何重写带copy关键字的setter？

若想令自己所写的对象具有拷贝功能, 则需实现 NSCopying 协议。

如果自定义的对象分为可变版本与不可变版本, 那么就要同时实现 NSCopying 与 NSMutableCopying 协议。

具体步骤：

需声明该类遵从 NSCopying 协议

实现 NSCopying 协议。该协议只有一个方法：



```
- (id)copyWithZone:(NSZone *)zone;
```

注意：一提到让自己的类用 copy 修饰符，我们总是想覆写 copy 方法，其实真正需要实现的却是 “copyWithZone” 方法。

- 重写带 copy 关键字的 setter，例如：



```
- (void)setName:(NSString *)name {  
    //[_name release];  
    _name = [name copy];  
}
```

3.深拷贝与浅拷贝分别是什么？

浅拷贝只是对指针的拷贝，拷贝后两个指针指向同一个内存空间，深拷贝不但对指针进行拷贝，

而且对指针指向的内容进行拷贝，经深拷贝后的指针是指向两个不同地址的指针。

当对象中存在指针成员时，除了在复制对象时需要考虑自定义拷贝构造函数，还应该考虑以下两种情形：

- 当函数的参数为对象时，实参传递给形参的实际上是实参的一个拷贝对象，系统自动通过拷贝构造函数实现；

- 当函数的返回值为一个对象时，该对象实际上是函数内对象的一个拷贝，用于返回函数调用处。

copy方法:如果是非可扩展类对象，则是浅拷贝。如果是可扩展类对象，则是深拷贝。

mutableCopy方法:无论是可扩展类对象还是不可扩展类对象，都是深拷贝。

4.@property的本质是什么？ivar、getter、setter是如何生成并添加到这个类中的？

@property 的本质是实例变量（ivar）+存取方法（access method = getter + setter），即 @property = ivar + getter + setter;

“属性” (property)作为 Objective-C 的一项特性，主要的作用就在于封装对象中的数据。Objective-C 对象通常会把其所需要的数据保存为各种实例变量。

实例变量一般通过“存取方法”(access method)来访问。其中，“获取方法”(getter)用于读取变量值，而“设置方法”(setter)用于写入变量值。

ivar、getter、setter 是自动合成这个类中的

完成属性定义后，编译器会自动编写访问这些属性所需的方法，此过程叫做“自动合成”(autosynthesis)。

需要强调的是，这个过程由编译器在编译期执行，所以编辑器里看不到这些“合成方法”(synthesized method)的源代码。

除了生成方法代码 getter、setter 之外，编译器还要自动向类中添加适当类型的实例变量，并且在属性名前面加下划线，以此作为实例变量的名字。

在前例中，会生成两个实例变量，其名称分别为 `_firstName` 与 `_lastName`。也可以在类的实现代码里通过 `@synthesize` 语法来指定实例变量的名字。

5.@protocol和category中如何使用@property

- 在 protocol 中使用 property 只会生成 setter 和 getter 方法声明,我们使用属性的目的,是希望遵守我协议的对象能实现该属性
- category 使用 @property 也是只会生成 setter 和 getter 方法的声明,如果我们真的需要给 category 增加属性的实现,需要借助于运行时的两个函数:
objc_setAssociatedObject和objc_getAssociatedObject

6.使用CADisplayLink、NSTimer有什么注意点？BAD_ACCESS在什么情况下出现？

1.使用CADisplayLink、NSTimer有什么注意点？

CADisplayLink、NSTimer会造成循环引用，可以使用YYWeakProxy或者为CADisplayLink、NSTimer添加block方法解决循环引用

2.BAD_ACCESS在什么情况下出现？

访问了悬垂指针，比如对一个已经释放的对象执行了release、访问已经释放对象的成员变量或者发消息。死循环

7.iOS内存分区情况

- 栈区（Stack）

由编译器自动分配释放，存放函数的参数，局部变量的值等

栈是向低地址扩展的数据结构，是一块连续的内存区域

- 堆区（Heap）

由程序员分配释放

是向高地址扩展的数据结构，是不连续的内存区域

- 全局区

全局变量和静态变量的存储是放在一块的，初始化的全局变量和静态变量在一块区域，未初始化的全局变量和未初始化的静态变量在相邻的另一块区域

程序结束后由系统释放

- 常量区

常量字符串就是放在这里的

程序结束后由系统释放

- 代码区

存放函数体的二进制代码

注：

- 在 iOS 中，堆区的内存是应用程序共享的，堆中的内存分配是系统负责的
- 系统使用一个链表来维护所有已经分配的内存空间（系统仅仅记录，并不管理具体的内容）
- 变量使用结束后，需要释放内存，OC 中是判断引用计数是否为 0，如果是就说明没有任何变量使用该空间，那么系统将其回收
- 当一个 app 启动后，代码区、常量区、全局区大小就已经固定，因此指向这些区的指针不会产生崩溃性的错误。而堆区和栈区是时时刻刻变化的（堆的创建销毁，栈的弹入弹

出），所以当使用一个指针指向这个区里面的内存时，一定要注意内存是否已经被释放，否则会产生程序崩溃（也即是野指针报错）

8.iOS内存管理方式

- Tagged Pointer（小对象）

Tagged Pointer 专门用来存储小的对象，例如 NSNumber 和 NSDate

Tagged Pointer 指针的值不再是地址了，而是真正的值。所以，实际上它不再是一个对象了，它只是一个披着对象皮的普通变量而已。所以，它的内存并不存储在堆中，也不需要 malloc 和 free

在内存读取上有着 3 倍的效率，创建时比以前快 106 倍

objc_msgSend 能识别 Tagged Pointer，比如 NSNumber 的 intValue 方法，直接从指针提取数据

使用 Tagged Pointer 后，指针内存存储的数据变成了 Tag + Data，也就是将数据直接存储在了指针中

- NONPOINTER_ISA（指针中存放与该对象内存相关的信息）苹果将 isa 设计成了联合体，在 isa 中存储了与该对象相关的一些内存的信息，原因也如上面所说，并不需要 64 个二进制位全部都用来存储指针。

isa 的结构：



```
// x86_64 架构
struct {
    uintptr_t nonpointer      : 1; // 0:普通指针, 1:优化过, 使用位域存储更多信息
    uintptr_t has_assoc      : 1; // 对象是否含有或曾经含有关联引用
```

```

        uintptr_t has_cxx_dtor      : 1; // 表示是否有C++析构函数或0
C的dealloc
        uintptr_t shiftcls         : 44; // 存放着 Class、Meta-Class
对象的内存地址信息
        uintptr_t magic            : 6; // 用于在调试时分辨对象是否
未完成初始化
        uintptr_t weakly_referenced : 1; // 是否被弱引用指向
        uintptr_t deallocating      : 1; // 对象是否正在释放
        uintptr_t has_sidetable_rc  : 1; // 是否需要使用 sidetable
来存储引用计数
        uintptr_t extra_rc          : 8; // 引用计数能够用 8 个二进制
位存储时，直接存储在这里
};

// arm64 架构
struct {
    uintptr_t nonpointer           : 1; // 0:普通指针，1:优化过，使
用位域存储更多信息
    uintptr_t has_assoc            : 1; // 对象是否含有或曾经含有关
联引用
    uintptr_t has_cxx_dtor         : 1; // 表示是否有C++析构函数或0
C的dealloc
    uintptr_t shiftcls             : 33; // 存放着 Class、Meta-Class
对象的内存地址信息
    uintptr_t magic                : 6; // 用于在调试时分辨对象是否
未完成初始化
    uintptr_t weakly_referenced    : 1; // 是否被弱引用指向
    uintptr_t deallocating         : 1; // 对象是否正在释放
    uintptr_t has_sidetable_rc     : 1; // 是否需要使用 sidetable
来存储引用计数
    uintptr_t extra_rc             : 19; // 引用计数能够用 19 个二
进制位存储时，直接存储在这里
};

```

这里的 has_sidetable_rc 和 extra_rc，has_sidetable_rc 表明该指针是否引用了 sidetable 散列表，之所以有这个选项，是因为少量的引用计数是不会直接存放在 SideTables 表中的，对象的引用计数会先存放在 extra_rc 中，当其被存满时，才会存入相应的 SideTables 散列表中，SideTables 中有很多张 SideTable，每个 SideTable 也都是一个散列表，而引用计数表就包含在 SideTable 之中。

- 散列表（引用计数表、弱引用表）

引用计数要么存放在 isa 的 extra_rc 中，要么存放在引用计数表中，而引用计数表包含在一个叫 SideTable 的结构中，它是一个散列表，也就是哈希表。而 SideTable 又包含在一个全局的 StripeMap 的哈希映射表中，这个表的名字叫 SideTables。

当一个对象访问 **SideTables** 时：

- 首先会取得对象的地址，将地址进行哈希运算，与 SideTables 中 SideTable 的个数取余，最后得到的结果就是该对象所要访问的 SideTable
- 在取得的 SideTable 中的 RefcountMap 表中再一次哈希查找，找到该对象在引用计数表中对应的位置
- 如果该位置存在对应的引用计数，则对其进行操作，如果没有对应的引用计数，则创建一个对应的 size_t 对象，其实就是一个 uint 类型的无符号整型
- 弱引用表也是一张哈希表的结构，其内部包含了每个对象对应的弱引用表 weak_entry_t，而 weak_entry_t 是一个结构体数组，其中包含的则是每一个对象弱引用的对象所对应的弱引用指针。

9.循环引用

循环引用的实质：多个对象相互之间有强引用，不能释放让系统回收。

如何解决循环引用？

1、避免产生循环引用，通常是将 strong 引用改为 weak 引用。比如在修饰属性时用weak 在 block内调用对象方法时，使用其弱引用，这里可以使用两个宏



```
#define WS(weakSelf) __weak __typeof(&*self)weakSelf = self; //
```

/ 弱引用



```
#define ST(strongSelf) __strong __typeof(&*self)strongSelf = weakSelf;
```

//使用这个要先声明weakSelf 还可以使用__block来修饰变量 在MRC下，__block不会增加其引用计数，避免了循环引用 在ARC下，__block修饰对象会被强引用，无法避免循环引用，需要手动解除。

2、在合适时机去手动断开循环引用。通常我们使用第一种。

- 代理(delegate)循环引用属于相互循环引用

delegate 是iOS中开发中比较常遇到的循环引用，一般在声明delegate的时候都要使用弱引用weak,或者assign,当然怎么选择使用assign还是weak，MRC的话只能用assign，在ARC的情况下最好使用weak，因为weak修饰的变量在释放后自动指向nil，防止野指针存在

- NSTimer循环引用属于相互循环使用

在控制器内，创建NSTimer作为其属性，由于定时器创建后也会强引用该控制器对象，那么该对象和定时器就相互循环引用了。如何解决呢？这里我们可以使用手动断开循环引用：如果是不重复定时器，在回调方法里将定时器invalidate并置为nil即可。如果是重复定时器，在合适的位置将其invalidate并置为nil即可

3、block循环引用

一个简单的例子：



```
@property (copy, nonatomic) dispatch_block_t myBlock;  
@property (copy, nonatomic) NSString *blockString;  
  
- (void)testBlock {  
    self.myBlock = ^() {  
        NSLog(@"%@", self.blockString);  
    };  
}
```

```
    };  
}
```

由于block会对block中的对象进行持有操作,就相当于持有了其中的对象,而如果此时block中的对象又持有了该block,则会造成循环引用。解决方案就是使用__weak修饰self即可



```
__weak typeof(self) weakSelf = self;  
  
self.myBlock = ^() {  
    NSLog(@"%@", weakSelf.blockString);  
};
```

并不是所有block都会造成循环引用。只有被强引用了的block才会产生循环引用 而比如 dispatch_async(dispatch_get_main_queue(), ^{}),[UIView animateWithDuration:1 animations:^(^{})]这些系统方法等 或者block并不是其属性而是临时变量,即栈block



```
[self testWithBlock:^(  
    NSLog(@"%@", self);  
)];  
  
- (void)testWithBlock:(dispatch_block_t)block {  
    block();  
}
```

还有一种场景,在block执行开始时self对象还未被释放,而执行过程中, self被释放了,由于是用weak修饰的,那么weakSelf也被释放了,此时在block里访问weakSelf时,就可能会发生错误(向nil对象发消息并不会崩溃,但也没任何效果)。

对于这种场景,应该在block中对 对象使用__strong修饰,使得在block期间对 对象持有,block执行结束后,解除其持有。



```
__weak typeof(self) weakSelf = self;

self.myBlock = ^() {

    __strong __typeof(self) strongSelf = weakSelf;

    [strongSelf test];

};
```

10.ARC 的 retainCount 怎么存储的？

ARC 的 retainCount 怎么存储的？

存在64张哈希表中，根据哈希算法去查找所在的位置，无需遍历，十分快捷

散列表（引用计数表、weak表）

- SideTables 表在 非嵌入式的64位系统中，有 64张 SideTable 表
- 每一张 SideTable 主要是由三部分组成。自旋锁、引用计数表、弱引用表。
- 全局的 引用计数 之所以不存在同一张表中，是为了避免资源竞争，解决效率的问题。
- 引用计数表 中引入了 分离锁的概念，将一张表分拆成多个部分，对他们分别加锁，可以实现并发操作，

提升执行效率

- 引用计数表（哈希表）
- 通过指针的地址，查找到引用计数的地址，大大提升查找效率
- 通过 DisguisedPtr(objc_object) 函数存储，同时也通过这个函数查找，这样就避免了循环遍历。

11. `ARC` 在编译时做了哪些工作？

根据代码执行的上下文语境，在适当的位置插入 retain, release

部分总结的内存管理面试题就到这里了，
以上内容对你有帮助的话，记得关注我们！

文章来源于网络，已尽可能标明作者以及来源，文章内容为作者独立观点，不代表本公众号立场，因文章侵权本公众号不承担任何法律及连带责任，如有侵权，请联系我们删除。



觉得不错的话，别忘了点个"在看"哦~ 📌

收录于话题 #BAT大厂面试题合集·9个

上一篇

iOS面试题合集（Runtime篇）

下一篇

iOS面试题合集（算法篇）

[阅读原文](#)

喜欢此内容的人还喜欢

iOS技术人员如何做晋升答辩？

iOSer

被忽略的「镇魂塔」，被遗忘的重要日子。

视觉志

怎么办？！这七款要出的皮肤，不买心里过不去，买了钱包受不了！

仙友Game